Credit Card Fraud Detection Using Deep Learning

This notebook compares MLP, 1D CNN, LSTM and Autoencoder models for highly imbalanced credit-card fraud detection. MLP/CNN include BCE, class-weighted BCE and Focal Loss; LSTM uses BCE. A Logistic Regression model is included only as a conventional reference baseline.

Experiments: stratified train/validation/test split, duplicate diagnostics, deduplicated rerun, chronological holdout, and five-seed robustness evaluation.

Important limitations: the dataset has no customer/card identifiers. CNN/LSTM reshape the 30 tabular features into pseudo-sequences; they do not learn customer transaction histories. V1–V28 are anonymized PCA components. `Time` is elapsed seconds, not a calendar timestamp.

1. Install optional explainability packages

```
!pip -q install shap lime
```

```
  Installing build dependencies ... done
  Getting requirements to build wheel ... done
  Preparing metadata (pyproject.toml) ... done
  Building wheel for lime (pyproject.toml) ... done
```

2. Imports and reproducibility

```
import os, random, warnings
warnings.filterwarnings("ignore")
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
import tensorflow as tf

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.utils.class_weight import compute_class_weight
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import (precision_score, recall_score, f1_score,
    average_precision_score, roc_auc_score, confusion_matrix,
    classification_report, precision_recall_curve)

SEED = 42
EPOCHS = 15
BATCH_SIZE = 512

def set_seed(seed):
    os.environ["PYTHONHASHSEED"] = str(seed)
    random.seed(seed)
    np.random.seed(seed)
    tf.keras.utils.set_random_seed(seed)

set_seed(SEED)
print("TensorFlow:", tf.__version__)
print("GPU:", tf.config.list_physical_devices("GPU"))
```

```
TensorFlow: 2.21.0
GPU: []
```

3. Upload and inspect dataset

```
from google.colab import files
uploaded = files.upload()
csv_files = [n for n in uploaded if n.lower().endswith(".csv")]
if not csv_files:
    raise ValueError("Upload creditcard.csv (CSV file, not ZIP).")
csv_name = next((n for n in csv_files if "creditcard" in n.lower()), csv_files[0])
df = pd.read_csv(csv_name)

required = {"Class", "Time", "Amount"}
if not required.issubset(df.columns):
    raise ValueError(f"Missing required columns: {required - set(df.columns)}")
if df.isnull().sum().sum():
```

```
    raise ValueError("Dataset contains missing values; inspect and handle them first.")

print("File:", csv_name, "| Shape:", df.shape)
print("Class counts:\n", df["Class"].value_counts())
print("Fraud percentage:", round(100 * df["Class"].mean(), 4))
display(df.head())
```

```
Choose files creditcard.csv
creditcard.csv(text/csv) - 150828752 bytes, last modified: 20/09/2019 - 100% done
Saving creditcard.csv to creditcard.csv
File: creditcard.csv | Shape: (284807, 31)
Class counts:
 Class
0    284315
1       492
Name: count, dtype: int64
Fraud percentage: 0.1727
```

	Time	V1	V2	V3	V4	V5	V6	V7	V8	V9	...	V21	V22
0	0.0	-1.359807	-0.072781	2.536347	1.378155	-0.338321	0.462388	0.239599	0.098698	0.363787	...	-0.018307	0.277838
1	0.0	1.191857	0.266151	0.166480	0.448154	0.060018	-0.082361	-0.078803	0.085102	-0.255425	...	-0.225775	-0.638672
2	1.0	-1.358354	-1.340163	1.773209	0.379780	-0.503198	1.800499	0.791461	0.247676	-1.514654	...	0.247998	0.771679
3	1.0	-0.966272	-0.185226	1.792993	-0.863291	-0.010309	1.247203	0.237609	0.377436	-1.387024	...	-0.108300	0.005274
4	2.0	-1.158233	0.877737	1.548718	0.403034	-0.407193	0.095921	0.592941	-0.270533	0.817739	...	-0.009431	0.798278

5 rows × 31 columns

4. Define architectures, preprocessing, thresholds and metrics

```
def focal_loss(gamma=2.0, alpha=0.75):
    def loss(y_true, y_pred):
        yt = tf.cast(tf.reshape(y_true, (-1, 1)), tf.float32)
        yp = tf.clip_by_value(tf.cast(y_pred, tf.float32), 1e-7, 1-1e-7)
        pt = yt*yp + (1-yt)*(1-yp)
        at = yt*alpha + (1-yt)*(1-alpha)
        return tf.reduce_mean(-at*tf.pow(1-pt, gamma)*tf.math.log(pt))
    return loss

def build_mlp(n):
    return tf.keras.Sequential([
        tf.keras.layers.Input((n,)),
        tf.keras.layers.Dense(64, activation="relu"),
        tf.keras.layers.Dropout(.30),
        tf.keras.layers.Dense(32, activation="relu"),
        tf.keras.layers.Dropout(.20),
        tf.keras.layers.Dense(1, activation="sigmoid")])

def build_cnn(shape):
    return tf.keras.Sequential([
        tf.keras.layers.Input(shape),
        tf.keras.layers.Conv1D(64, 3, padding="same", activation="relu"),
        tf.keras.layers.MaxPooling1D(2),
        tf.keras.layers.Conv1D(32, 3, padding="same", activation="relu"),
        tf.keras.layers.GlobalMaxPooling1D(),
        tf.keras.layers.Dense(32, activation="relu"),
        tf.keras.layers.Dropout(.30),
        tf.keras.layers.Dense(1, activation="sigmoid")])

def build_lstm(shape):
    return tf.keras.Sequential([
        tf.keras.layers.Input(shape),
        tf.keras.layers.LSTM(32, dropout=.20),
        tf.keras.layers.Dense(16, activation="relu"),
        tf.keras.layers.Dropout(.20),
        tf.keras.layers.Dense(1, activation="sigmoid")])

def compile_model(model, loss="binary_crossentropy"):
    model.compile(optimizer=tf.keras.optimizers.Adam(1e-3), loss=loss)
    return model

def callbacks():
    return [tf.keras.callbacks.EarlyStopping(
        monitor="val_loss", patience=3, restore_best_weights=True)]

def choose_threshold(y, scores):
    p, r, t = precision_recall_curve(y, scores)
    if len(t) == 0:
```

```
        return 0.5
    p, r = p[:-1], r[:-1]
    f1 = 2*p*r/(p+r+1e-12)
    return float(t[int(np.argmax(f1))])

def metrics_at_threshold(y, scores, threshold):
    pred = (scores >= threshold).astype(int)
    return {
        "Threshold": float(threshold),
        "Precision": precision_score(y, pred, zero_division=0),
        "Recall": recall_score(y, pred, zero_division=0),
        "F1": f1_score(y, pred, zero_division=0),
        "PR-AUC": average_precision_score(y, scores),
        "ROC-AUC": roc_auc_score(y, scores)
    }, pred

def prepare_arrays(train_df, val_df, test_df):
    Xtr = train_df.drop(columns="Class").copy()
    Xv = val_df.drop(columns="Class").copy()
    Xt = test_df.drop(columns="Class").copy()
    ytr = train_df["Class"].to_numpy(dtype=np.int32)
    yv = val_df["Class"].to_numpy(dtype=np.int32)
    yt = test_df["Class"].to_numpy(dtype=np.int32)
    scaler = StandardScaler()
    cols = ["Time", "Amount"]
    Xtr.loc[:, cols] = scaler.fit_transform(Xtr[cols])
    Xv.loc[:, cols] = scaler.transform(Xv[cols])
    Xt.loc[:, cols] = scaler.transform(Xt[cols])
    return (Xtr.to_numpy(np.float32), Xv.to_numpy(np.float32),
            Xt.to_numpy(np.float32), ytr, yv, yt, Xtr.columns.tolist())

def train_autoencoder(Xtr, Xv, ytr, yv, seed):
    set_seed(seed)
    inp = tf.keras.layers.Input(shape=(Xtr.shape[1],))
    z = tf.keras.layers.Dense(16, activation="relu")(inp)
    z = tf.keras.layers.Dense(8, activation="relu")(z)
    z = tf.keras.layers.Dense(16, activation="relu")(z)
    out = tf.keras.layers.Dense(Xtr.shape[1], activation="linear")(z)
    ae = tf.keras.Model(inp, out)
    ae.compile(optimizer="adam", loss="mse")
    ae.fit(Xtr[ytr==0], Xtr[ytr==0],
           validation_data=(Xv[yv==0], Xv[yv==0]),
           epochs=20, batch_size=BATCH_SIZE, callbacks=callbacks(), verbose=0)
    return ae

def reconstruction_errors(model, X):
    rec = model.predict(X, batch_size=BATCH_SIZE, verbose=0)
    return np.mean(np.square(X-rec), axis=1)
```

5. Stratified split and model training pipeline

```
def stratified_split(data, seed):
    train, temp = train_test_split(data, test_size=.40, stratify=data["Class"], random_state=seed)
    val, test = train_test_split(temp, test_size=.50, stratify=temp["Class"], random_state=seed)
    return train.copy(), val.copy(), test.copy()

def run_stratified_pipeline(data, seed=42, verbose=0):
    set_seed(seed)
    tr, va, te = stratified_split(data, seed)
    Xtr, Xv, Xt, ytr, yv, yt, feature_names = prepare_arrays(tr, va, te)
    Xtr_seq, Xv_seq, Xt_seq = [x.reshape(-1, x.shape[1], 1) for x in (Xtr, Xv, Xt)]
    cw = compute_class_weight(class_weight="balanced", classes=np.array([0,1]), y=ytr)
    class_weights = {0: float(cw[0]), 1: float(cw[1])}
    model_specs = {
        "MLP - BCE": (build_mlp(Xtr.shape[1]), "binary_crossentropy", None, Xtr, Xv, Xt),
        "MLP - Weighted BCE": (build_mlp(Xtr.shape[1]), "binary_crossentropy", class_weights, Xtr, Xv, Xt),
        "MLP - Focal Loss": (build_mlp(Xtr.shape[1]), focal_loss(), None, Xtr, Xv, Xt),
        "CNN - BCE": (build_cnn((Xtr.shape[1],1)), "binary_crossentropy", None, Xtr_seq, Xv_seq, Xt_seq),
        "CNN - Weighted BCE": (build_cnn((Xtr.shape[1],1)), "binary_crossentropy", class_weights, Xtr_seq, Xv_se
        "CNN - Focal Loss": (build_cnn((Xtr.shape[1],1)), focal_loss(), None, Xtr_seq, Xv_seq, Xt_seq),
        "LSTM - BCE": (build_lstm((Xtr.shape[1],1)), "binary_crossentropy", None, Xtr_seq, Xv_seq, Xt_seq)
    }
    models, val_scores, test_scores, rows, thresholds = {}, {}, {}, [], {}
    for name, (model, loss, weights, a, b, c) in model_specs.items():
        set_seed(seed)
        compile_model(model, loss)
        fit_args = {}
        if weights is not None:
            fit_args["class_weight"] = weights
        model.fit(a, ytr, validation_data=(b,yv), epochs=EPOCHS,
```

```
                    batch_size=BATCH_SIZE, callbacks=callbacks(), verbose=verbose, **fit_args)
            pv = model.predict(b, batch_size=BATCH_SIZE, verbose=0).ravel()
            pt = model.predict(c, batch_size=BATCH_SIZE, verbose=0).ravel()
            th = choose_threshold(yv, pv)
            row, _ = metrics_at_threshold(yt, pt, th)
            row.update({"Model":name, "Validation PR-AUC":average_precision_score(yv,pv)})
            rows.append(row)
            models[name] = model
            val_scores[name], test_scores[name], thresholds[name] = pv, pt, th

        ae = train_autoencoder(Xtr, Xv, ytr, yv, seed)
        av, at = reconstruction_errors(ae, Xv), reconstruction_errors(ae, Xt)
        th = choose_threshold(yv, av)
        row, _ = metrics_at_threshold(yt, at, th)
        row.update({"Model":"Autoencoder", "Validation PR-AUC":average_precision_score(yv,av)})
        rows.append(row)
        models["Autoencoder"] = ae
        val_scores["Autoencoder"], test_scores["Autoencoder"], thresholds["Autoencoder"] = av, at, th

        # Conventional reference baseline, not a deep-learning model.
        lr = LogisticRegression(class_weight="balanced", max_iter=2000, solver="lbfgs", random_state=seed)
        lr.fit(Xtr, ytr)
        lv, lt = lr.predict_proba(Xv)[:,1], lr.predict_proba(Xt)[:,1]
        th = choose_threshold(yv, lv)
        row, _ = metrics_at_threshold(yt, lt, th)
        row.update({"Model":"Logistic Regression (reference baseline)",
                    "Validation PR-AUC":average_precision_score(yv,lv)})
        rows.append(row)
        models["Logistic Regression (reference baseline)"] = lr
        val_scores["Logistic Regression (reference baseline)"], test_scores["Logistic Regression (reference baseline
        thresholds["Logistic Regression (reference baseline)"] = th

        table = pd.DataFrame(rows).sort_values("Validation PR-AUC", ascending=False).reset_index(drop=True)
        return {"table":table, "models":models, "val_scores":val_scores,
                "test_scores":test_scores, "thresholds":thresholds,
                "split":(tr,va,te), "arrays":(Xtr,Xv,Xt,ytr,yv,yt,feature_names)}
```

6. Main stratified experiment: original dataset

```
original_run = run_stratified_pipeline(df, seed=SEED, verbose=1)
original_results = original_run["table"]
display(original_results)
original_results.to_csv("original_stratified_results.csv", index=False)

best_model_name = original_results.iloc[0]["Model"]
print("Selected using validation PR-AUC:", best_model_name)
best_row = original_results.iloc[0]
print("\nSelected model test metrics (reported once):")
print(best_row[["Precision","Recall","F1","PR-AUC","ROC-AUC"]])
```

```
Epoch 2/15
334/334 ━━━━━━━━━━━━━━━━━━━━ 3s 5ms/step - loss: 0.0051 - val_loss: 0.0052
Epoch 3/15
334/334 ━━━━━━━━━━━━━━━━━━━━ 1s 4ms/step - loss: 0.0042 - val_loss: 0.0042
Epoch 4/15
334/334 ━━━━━━━━━━━━━━━━━━━━ 1s 4ms/step - loss: 0.0039 - val_loss: 0.0039
Epoch 5/15
334/334 ━━━━━━━━━━━━━━━━━━━━ 1s 4ms/step - loss: 0.0036 - val_loss: 0.0039
Epoch 6/15
334/334 ━━━━━━━━━━━━━━━━━━━━ 1s 4ms/step - loss: 0.0033 - val_loss: 0.0037
Epoch 7/15
334/334 ━━━━━━━━━━━━━━━━━━━━ 1s 4ms/step - loss: 0.0030 - val_loss: 0.0037
Epoch 8/15
334/334 ━━━━━━━━━━━━━━━━━━━━ 1s 4ms/step - loss: 0.0028 - val_loss: 0.0037
Epoch 9/15
334/334 ━━━━━━━━━━━━━━━━━━━━ 1s 4ms/step - loss: 0.0029 - val_loss: 0.0036
Epoch 10/15
334/334 ━━━━━━━━━━━━━━━━━━━━ 1s 4ms/step - loss: 0.0029 - val_loss: 0.0037
Epoch 11/15
334/334 ━━━━━━━━━━━━━━━━━━━━ 2s 6ms/step - loss: 0.0027 - val_loss: 0.0036
Epoch 12/15
334/334 ━━━━━━━━━━━━━━━━━━━━ 2s 5ms/step - loss: 0.0027 - val_loss: 0.0036
Epoch 13/15
334/334 ━━━━━━━━━━━━━━━━━━━━ 1s 4ms/step - loss: 0.0025 - val_loss: 0.0037
Epoch 14/15
334/334 ━━━━━━━━━━━━━━━━━━━━ 1s 4ms/step - loss: 0.0023 - val_loss: 0.0036
Epoch 15/15
334/334 ━━━━━━━━━━━━━━━━━━━━ 1s 4ms/step - loss: 0.0023 - val_loss: 0.0037
Epoch 1/15
334/334 ━━━━━━━━━━━━━━━━━━━━ 2s 4ms/step - loss: 0.2938 - val_loss: 0.1198
Epoch 2/15
334/334 ━━━━━━━━━━━━━━━━━━━━ 1s 4ms/step - loss: 0.1579 - val_loss: 0.0988
Epoch 3/15
334/334 ━━━━━━━━━━━━━━━━━━━━ 1s 4ms/step - loss: 0.1503 - val_loss: 0.1091
Epoch 4/15
334/334 ━━━━━━━━━━━━━━━━━━━━ 2s 6ms/step - loss: 0.1206 - val_loss: 0.0933
Epoch 5/15
334/334 ━━━━━━━━━━━━━━━━━━━━ 2s 5ms/step - loss: 0.1229 - val_loss: 0.0932
Epoch 6/15
334/334 ━━━━━━━━━━━━━━━━━━━━ 2s 4ms/step - loss: 0.1134 - val_loss: 0.0736
Epoch 7/15
334/334 ━━━━━━━━━━━━━━━━━━━━ 1s 4ms/step - loss: 0.1002 - val_loss: 0.0656
Epoch 8/15
334/334 ━━━━━━━━━━━━━━━━━━━━ 1s 4ms/step - loss: 0.1104 - val_loss: 0.0844
Epoch 9/15
334/334 ━━━━━━━━━━━━━━━━━━━━ 1s 4ms/step - loss: 0.1119 - val_loss: 0.0908
Epoch 10/15
334/334 ━━━━━━━━━━━━━━━━━━━━ 1s 4ms/step - loss: 0.0921 - val_loss: 0.0676
Epoch 1/15
334/334 ━━━━━━━━━━━━━━━━━━━━ 3s 5ms/step - loss: 0.0031 - val_loss: 0.0012
Epoch 2/15
334/334 ━━━━━━━━━━━━━━━━━━━━ 3s 5ms/step - loss: 9.7598e-04 - val_loss: 9.0140e-04
Epoch 3/15
334/334 ━━━━━━━━━━━━━━━━━━━━ 1s 4ms/step - loss: 8.0475e-04 - val_loss: 8.1111e-04
Epoch 4/15
334/334 ━━━━━━━━━━━━━━━━━━━━ 1s 4ms/step - loss: 6.8384e-04 - val_loss: 7.6327e-04
Epoch 5/15
334/334 ━━━━━━━━━━━━━━━━━━━━ 1s 4ms/step - loss: 6.1602e-04 - val_loss: 7.1912e-04
Epoch 6/15
334/334 ━━━━━━━━━━━━━━━━━━━━ 1s 4ms/step - loss: 6.0302e-04 - val_loss: 6.7074e-04
Epoch 7/15
334/334 ━━━━━━━━━━━━━━━━━━━━ 1s 4ms/step - loss: 5.2429e-04 - val_loss: 6.4087e-04
Epoch 8/15
334/334 ━━━━━━━━━━━━━━━━━━━━ 1s 4ms/step - loss: 5.1127e-04 - val_loss: 6.3897e-04
Epoch 9/15
334/334 ━━━━━━━━━━━━━━━━━━━━ 1s 4ms/step - loss: 5.0453e-04 - val_loss: 6.5698e-04
Epoch 10/15
334/334 ━━━━━━━━━━━━━━━━━━━━ 2s 5ms/step - loss: 5.0404e-04 - val_loss: 6.3610e-04
Epoch 11/15
334/334 ━━━━━━━━━━━━━━━━━━━━ 2s 6ms/step - loss: 4.5669e-04 - val_loss: 6.5640e-04
Epoch 12/15
334/334 ━━━━━━━━━━━━━━━━━━━━ 2s 4ms/step - loss: 4.2796e-04 - val_loss: 6.7317e-04
Epoch 13/15
334/334 ━━━━━━━━━━━━━━━━━━━━ 1s 4ms/step - loss: 3.7234e-04 - val_loss: 6.9297e-04
Epoch 1/15
334/334 ━━━━━━━━━━━━━━━━━━━━ 12s 31ms/step - loss: 0.0382 - val_loss: 0.0095
Epoch 2/15
334/334 ━━━━━━━━━━━━━━━━━━━━ 9s 26ms/step - loss: 0.0072 - val_loss: 0.0059
Epoch 3/15
334/334 ━━━━━━━━━━━━━━━━━━━━ 10s 30ms/step - loss: 0.0050 - val_loss: 0.0046
Epoch 4/15
334/334 ━━━━━━━━━━━━━━━━━━━━ 11s 32ms/step - loss: 0.0042 - val_loss: 0.0045
Epoch 5/15
334/334 ━━━━━━━━━━━━━━━━━━━━ 10s 30ms/step - loss: 0.0040 - val_loss: 0.0043
Epoch 6/15
334/334 ━━━━━━━━━━━━━━━━━━━━ 9s 28ms/step - loss: 0.0037 - val_loss: 0.0045
Epoch 7/15
334/334 ━━━━━━━━━━━━━━━━━━━━ 11s 31ms/step - loss: 0.0037 - val_loss: 0.0044
```

```
Epoch 8/15
334/334 ━━━━━━━━━━━━━━━━━━━━ 10s 31ms/step - loss: 0.0036 - val_loss: 0.0040
Epoch 9/15
334/334 ━━━━━━━━━━━━━━━━━━━━ 10s 30ms/step - loss: 0.0035 - val_loss: 0.0040
Epoch 10/15
334/334 ━━━━━━━━━━━━━━━━━━━━ 9s 26ms/step - loss: 0.0034 - val_loss: 0.0038
Epoch 11/15
334/334 ━━━━━━━━━━━━━━━━━━━━ 11s 32ms/step - loss: 0.0033 - val_loss: 0.0039
Epoch 12/15
334/334 ━━━━━━━━━━━━━━━━━━━━ 10s 30ms/step - loss: 0.0032 - val_loss: 0.0040
Epoch 13/15
334/334 ━━━━━━━━━━━━━━━━━━━━ 8s 25ms/step - loss: 0.0032 - val_loss: 0.0036
Epoch 14/15
334/334 ━━━━━━━━━━━━━━━━━━━━ 10s 26ms/step - loss: 0.0031 - val_loss: 0.0037
Epoch 15/15
334/334 ━━━━━━━━━━━━━━━━━━━━ 12s 30ms/step - loss: 0.0030 - val_loss: 0.0039
Epoch 1/15
334/334 ━━━━━━━━━━━━━━━━━━━━ 12s 33ms/step - loss: 0.3788 - val_loss: 0.3402
Epoch 2/15
334/334 ━━━━━━━━━━━━━━━━━━━━ 10s 30ms/step - loss: 0.2590 - val_loss: 0.2338
Epoch 3/15
334/334 ━━━━━━━━━━━━━━━━━━━━ 10s 29ms/step - loss: 0.2445 - val_loss: 0.2483
Epoch 4/15
334/334 ━━━━━━━━━━━━━━━━━━━━ 10s 27ms/step - loss: 0.2042 - val_loss: 0.1570
Epoch 5/15
334/334 ━━━━━━━━━━━━━━━━━━━━ 11s 29ms/step - loss: 0.1922 - val_loss: 0.1086
Epoch 6/15
334/334 ━━━━━━━━━━━━━━━━━━━━ 17s 48ms/step - loss: 0.1760 - val_loss: 0.1308
Epoch 7/15
334/334 ━━━━━━━━━━━━━━━━━━━━ 16s 49ms/step - loss: 0.1652 - val_loss: 0.1053
Epoch 8/15
334/334 ━━━━━━━━━━━━━━━━━━━━ 14s 28ms/step - loss: 0.1538 - val_loss: 0.1510
Epoch 9/15
334/334 ━━━━━━━━━━━━━━━━━━━━ 9s 28ms/step - loss: 0.1522 - val_loss: 0.1016
Epoch 10/15
334/334 ━━━━━━━━━━━━━━━━━━━━ 10s 30ms/step - loss: 0.1489 - val_loss: 0.1023
Epoch 11/15
334/334 ━━━━━━━━━━━━━━━━━━━━ 11s 32ms/step - loss: 0.1317 - val_loss: 0.0788
Epoch 12/15
334/334 ━━━━━━━━━━━━━━━━━━━━ 9s 27ms/step - loss: 0.1303 - val_loss: 0.2773
Epoch 13/15
334/334 ━━━━━━━━━━━━━━━━━━━━ 11s 32ms/step - loss: 0.1345 - val_loss: 0.0710
Epoch 14/15
334/334 ━━━━━━━━━━━━━━━━━━━━ 10s 30ms/step - loss: 0.1201 - val_loss: 0.0586
Epoch 15/15
334/334 ━━━━━━━━━━━━━━━━━━━━ 10s 30ms/step - loss: 0.1513 - val_loss: 0.2663
Epoch 1/15
334/334 ━━━━━━━━━━━━━━━━━━━━ 12s 31ms/step - loss: 0.0029 - val_loss: 0.0011
Epoch 2/15
334/334 ━━━━━━━━━━━━━━━━━━━━ 9s 27ms/step - loss: 9.6899e-04 - val_loss: 8.5817e-04
Epoch 3/15
334/334 ━━━━━━━━━━━━━━━━━━━━ 11s 32ms/step - loss: 7.7268e-04 - val_loss: 8.4377e-04
Epoch 4/15
334/334 ━━━━━━━━━━━━━━━━━━━━ 10s 30ms/step - loss: 7.3630e-04 - val_loss: 8.0917e-04
Epoch 5/15
334/334 ━━━━━━━━━━━━━━━━━━━━ 10s 30ms/step - loss: 6.9759e-04 - val_loss: 7.9930e-04
Epoch 6/15
334/334 ━━━━━━━━━━━━━━━━━━━━ 9s 26ms/step - loss: 6.3958e-04 - val_loss: 7.8279e-04
Epoch 7/15
334/334 ━━━━━━━━━━━━━━━━━━━━ 11s 32ms/step - loss: 6.3084e-04 - val_loss: 7.9218e-04
Epoch 8/15
334/334 ━━━━━━━━━━━━━━━━━━━━ 11s 32ms/step - loss: 5.8842e-04 - val_loss: 7.3605e-04
Epoch 9/15
334/334 ━━━━━━━━━━━━━━━━━━━━ 9s 27ms/step - loss: 6.1407e-04 - val_loss: 7.4573e-04
Epoch 10/15
334/334 ━━━━━━━━━━━━━━━━━━━━ 10s 25ms/step - loss: 5.6002e-04 - val_loss: 7.3982e-04
Epoch 11/15
334/334 ━━━━━━━━━━━━━━━━━━━━ 10s 31ms/step - loss: 5.5990e-04 - val_loss: 7.6887e-04
Epoch 1/15
334/334 ━━━━━━━━━━━━━━━━━━━━ 21s 57ms/step - loss: 0.0509 - val_loss: 0.0070
Epoch 2/15
334/334 ━━━━━━━━━━━━━━━━━━━━ 18s 54ms/step - loss: 0.0083 - val_loss: 0.0063
Epoch 3/15
334/334 ━━━━━━━━━━━━━━━━━━━━ 17s 52ms/step - loss: 0.0078 - val_loss: 0.0048
Epoch 4/15
334/334 ━━━━━━━━━━━━━━━━━━━━ 19s 57ms/step - loss: 0.0069 - val_loss: 0.0045
Epoch 5/15
334/334 ━━━━━━━━━━━━━━━━━━━━ 19s 52ms/step - loss: 0.0061 - val_loss: 0.0044
Epoch 6/15
334/334 ━━━━━━━━━━━━━━━━━━━━ 17s 52ms/step - loss: 0.0062 - val_loss: 0.0046
Epoch 7/15
334/334 ━━━━━━━━━━━━━━━━━━━━ 19s 57ms/step - loss: 0.0061 - val_loss: 0.0046
Epoch 8/15
334/334 ━━━━━━━━━━━━━━━━━━━━ 17s 52ms/step - loss: 0.0061 - val_loss: 0.0044
Epoch 9/15
334/334 ━━━━━━━━━━━━━━━━━━━━ 17s 51ms/step - loss: 0.0059 - val_loss: 0.0042
Epoch 10/15
334/334 ━━━━━━━━━━━━━━━━━━━━ 22s 57ms/step - loss: 0.0067 - val_loss: 0.0041
Epoch 11/15
334/334 ━━━━━━━━━━━━━━━━━━━━ 17s 52ms/step - loss: 0.0063 - val_loss: 0.0042
Epoch 12/15
```

Next steps: Generate code with original_results | New interactive sheet

```
334/334 ━━━━━━━━━━━━━━━━━━━━ 18s 52ms/step - loss: 0.0063 - val_loss: 0.0041
Epoch 13/15
334/334 ━━━━━━━━━━━━━━━━━━━━ 19s 58ms/step - loss: 0.0063 - val_loss: 0.0041
Epoch 14/15
```

7. Duplicate-row diagnostics and deduplicated rerun

```
duplicate_count = int(df.duplicated().sum())
tr, va, te = original_run["split"]

def overlap_count(query, reference):
    ref = pd.MultiIndex.from_frame(reference)
    q = pd.MultiIndex.from_frame(query)
    return int(q.isin(ref).sum())

train_features = tr.copy()
val_features = va.copy()
test_features = te.copy()
print("Exact duplicate rows in full original dataset:", duplicate_count)
print("Validation rows with an exact feature duplicate in train:",
      overlap_count(val_features, train_features))
print("Test rows with an exact feature duplicate in train:",
      overlap_count(test_features, train_features))

df_dedup = df.drop_duplicates().reset_index(drop=True)
print("Rows before:", len(df), "| after deduplication:", len(df_dedup),
      "| removed:", len(df)-len(df_dedup))
dedup_run = run_stratified_pipeline(df_dedup, seed=SEED, verbose=0)
dedup_results = dedup_run["table"]

comparison = pd.concat([
    original_results.assign(Dataset="Original"),
    dedup_results.assign(Dataset="Deduplicated")
], ignore_index=True)
comparison.to_csv("original_vs_deduplicated_results.csv", index=False)
display(comparison[["Dataset","Model","Precision","Recall","F1","PR-AUC","ROC-AUC"]]
        .sort_values(["Model","Dataset"]))
```

```
Exact duplicate rows in full original dataset: 1081
Validation rows with an exact feature duplicate in train: 297
Test rows with an exact feature duplicate in train: 271
Rows before: 284807 | after deduplication: 283726 | removed: 1081
```

	Dataset	Model	Precision	Recall	F1	PR-AUC	ROC-AUC
17	Deduplicated	Autoencoder	0.464286	0.547368	0.502415	0.496669	0.964060
8	Original	Autoencoder	0.438849	0.616162	0.512605	0.473192	0.962842
12	Deduplicated	CNN - BCE	0.847059	0.757895	0.800000	0.757516	0.967698
2	Original	CNN - BCE	0.843750	0.818182	0.830769	0.806705	0.968007
11	Deduplicated	CNN - Focal Loss	0.884615	0.726316	0.797688	0.773348	0.961549
3	Original	CNN - Focal Loss	0.902439	0.747475	0.817680	0.790852	0.971792
16	Deduplicated	CNN - Weighted BCE	0.676768	0.705263	0.690722	0.641250	0.958363
7	Original	CNN - Weighted BCE	0.837209	0.727273	0.778378	0.719200	0.966831
15	Deduplicated	LSTM - BCE	0.840909	0.778947	0.808743	0.710446	0.955084
4	Original	LSTM - BCE	0.880435	0.818182	0.848168	0.765674	0.948479
13	Deduplicated	Logistic Regression (reference baseline)	0.887500	0.747368	0.811429	0.730846	0.966956
5	Original	Logistic Regression (reference baseline)	0.886364	0.787879	0.834225	0.757532	0.966117
9	Deduplicated	MLP - BCE	0.892857	0.789474	0.837989	0.818975	0.968444
0	Original	MLP - BCE	0.881720	0.828283	0.854167	0.843964	0.979354
10	Deduplicated	MLP - Focal Loss	0.852273	0.789474	0.819672	0.806879	0.969752
1	Original	MLP - Focal Loss	0.877778	0.797980	0.835979	0.840167	0.978063
14	Deduplicated	MLP - Weighted BCE	0.873563	0.800000	0.835165	0.730440	0.971543
6	Original	MLP - Weighted BCE	0.875000	0.777778	0.823529	0.726813	0.975387

8. Five-seed evaluation on the original stratified split

```
def run_seed_metrics(data, seed):
    run = run_stratified_pipeline(data, seed=seed, verbose=0)
    return run["table"][["Model","PR-AUC","Recall","Precision","F1"]]

seed_values = [42, 1, 2, 3, 4]
seed_frames = []
for s in seed_values:
```

```
    print("Running seed:", s)
    tmp = run_seed_metrics(df, s)
    tmp["Seed"] = s
    seed_frames.append(tmp)

seed_results = pd.concat(seed_frames, ignore_index=True)
summary = seed_results.groupby("Model")[["PR-AUC","Recall","Precision","F1"]].agg(["mean","std"])
summary.columns = [f"{a}_{b}" for a,b in summary.columns]
summary = summary.reset_index()
display(summary)
summary.to_csv("five_seed_mean_std.csv", index=False)
seed_results.to_csv("five_seed_raw_results.csv", index=False)
```

```
Running seed: 42
Running seed: 1
Running seed: 2
Running seed: 3
Running seed: 4
```

	Model	PR-AUC_mean	PR-AUC_std	Recall_mean	Recall_std	Precision_mean	Precision_std	F1_mean	F1_std
0	Autoencoder	0.511390	0.066433	0.595960	0.066621	0.490819	0.054657	0.535276	0.041406
1	CNN - BCE	0.823901	0.044087	0.806061	0.027104	0.858579	0.054929	0.830882	0.034345
2	CNN - Focal Loss	0.809734	0.035323	0.795960	0.044260	0.853928	0.078383	0.821860	0.044190
3	CNN - Weighted BCE	0.720147	0.061051	0.725253	0.066005	0.808159	0.067066	0.761888	0.045784
4	LSTM - BCE	0.753511	0.068699	0.789899	0.040023	0.872968	0.024399	0.828963	0.027714
5	Logistic Regression (reference baseline)	0.756816	0.056006	0.775758	0.045957	0.886880	0.021132	0.827099	0.030813
6	MLP - BCE	0.826686	0.049862	0.808081	0.035712	0.887289	0.023333	0.845444	0.023133

Next steps: Generate code with summary | New interactive sheet

9. Chronological holdout experiment

```
def chronological_split(data):
    ordered = data.sort_values("Time", kind="mergesort").reset_index(drop=True)
    n = len(ordered)
    return (ordered.iloc[:int(.60*n)].copy(),
            ordered.iloc[int(.60*n):int(.80*n)].copy(),
            ordered.iloc[int(.80*n):].copy())

def run_chronological(data, seed=42):
    set_seed(seed)
    tr, va, te = chronological_split(data)
    print("Chronological split fraud counts:",
          {"train":int(tr.Class.sum()),"validation":int(va.Class.sum()),"test":int(te.Class.sum())})
    Xtr,Xv,Xt,ytr,yv,yt,features = prepare_arrays(tr,va,te)
    seq = [x.reshape(-1,x.shape[1],1) for x in (Xtr,Xv,Xt)]
    specs = {
        "MLP - BCE":(build_mlp(Xtr.shape[1]),"binary_crossentropy",Xtr,Xv,Xt),
        "MLP - Focal Loss":(build_mlp(Xtr.shape[1]),focal_loss(),Xtr,Xv,Xt),
        "CNN - BCE":(build_cnn((Xtr.shape[1],1)),"binary_crossentropy",seq[0],seq[1],seq[2]),
        "LSTM - BCE":(build_lstm((Xtr.shape[1],1)),"binary_crossentropy",seq[0],seq[1],seq[2])
    }
    rows=[]
    for name,(model,loss,a,b,c) in specs.items():
        set_seed(seed)
        compile_model(model,loss)
        model.fit(a,ytr,validation_data=(b,yv),epochs=EPOCHS,batch_size=BATCH_SIZE,
                  callbacks=callbacks(),verbose=0)
        pv=model.predict(b,batch_size=BATCH_SIZE,verbose=0).ravel()
        pt=model.predict(c,batch_size=BATCH_SIZE,verbose=0).ravel()
        threshold=choose_threshold(yv,pv)
        row,_=metrics_at_threshold(yt,pt,threshold)
        row.update({"Model":name,"Validation PR-AUC":average_precision_score(yv,pv)})
        rows.append(row)

    ae=train_autoencoder(Xtr,Xv,ytr,yv,seed)
    av,at=reconstruction_errors(ae,Xv),reconstruction_errors(ae,Xt)
    threshold=choose_threshold(yv,av)
    row,_=metrics_at_threshold(yt,at,threshold)
    row.update({"Model":"Autoencoder","Validation PR-AUC":average_precision_score(yv,av)})
```

```
    rows.append(row)
    return pd.DataFrame(rows).sort_values("Validation PR-AUC",ascending=False).reset_index(drop=True)

chronological_results = run_chronological(df, SEED)
display(chronological_results)
chronological_results.to_csv("chronological_results.csv",index=False)
```

```
Chronological split fraud counts: {'train': 360, 'validation': 57, 'test': 75}
```

	Threshold	Precision	Recall	F1	PR-AUC	ROC-AUC	Model	Validation PR-AUC
0	0.443892	0.942308	0.653333	0.771654	0.805253	0.955400	MLP - BCE	0.784682
1	0.521446	0.898305	0.706667	0.791045	0.815886	0.980852	MLP - Focal Loss	0.769682
2	0.371322	0.816667	0.653333	0.725926	0.767974	0.954774	CNN - BCE	0.755465
3	0.376954	0.910714	0.680000	0.778626	0.758418	0.947786	LSTM - BCE	0.741040
4	6.540689	0.359551	0.426667	0.390244	0.234485	0.933045	Autoencoder	0.124080

Next steps: Generate code with chronological_results | New interactive sheet

10. Confusion matrix and training-independent model comparison

```
# Detailed test report for the validation-PR-AUC-selected model only.
best_name = original_results.iloc[0]["Model"]
_, _, y_test = original_run["arrays"][3:6]
best_scores = original_run["test_scores"][best_name]
best_threshold = original_run["thresholds"][best_name]
best_metrics, best_pred = metrics_at_threshold(y_test,best_scores,best_threshold)
print("Best model:",best_name)
print("Threshold selected on validation:",round(best_threshold,6))
print("\n",classification_report(y_test,best_pred,digits=4,zero_division=0))
print("Confusion matrix:\n",confusion_matrix(y_test,best_pred))

plt.figure(figsize=(5,4))
sns.heatmap(confusion_matrix(y_test,best_pred),annot=True,fmt="d",cmap="Blues")
plt.title("Selected model confusion matrix: "+best_name)
plt.xlabel("Predicted"); plt.ylabel("Actual"); plt.show()

original_results.set_index("Model")[["Validation PR-AUC","PR-AUC","Recall","Precision","F1"]].plot(
    kind="bar",figsize=(13,5),ylim=(0,1),rot=35)
plt.title("Stratified experiment: validation ranking and test metrics")
plt.tight_layout(); plt.show()
```

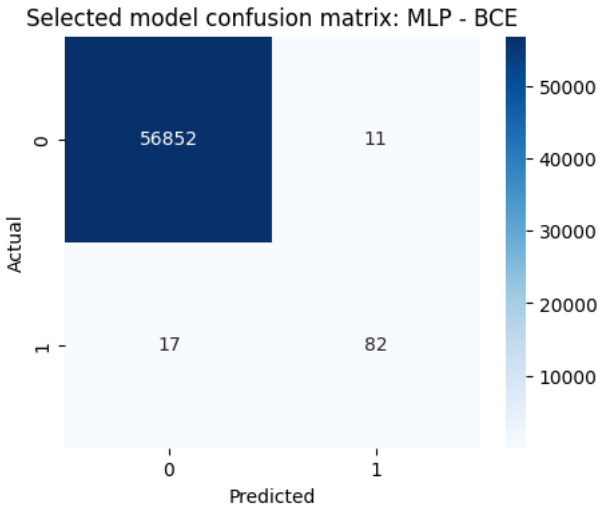
```
Best model: MLP - BCE
Threshold selected on validation: 0.659421

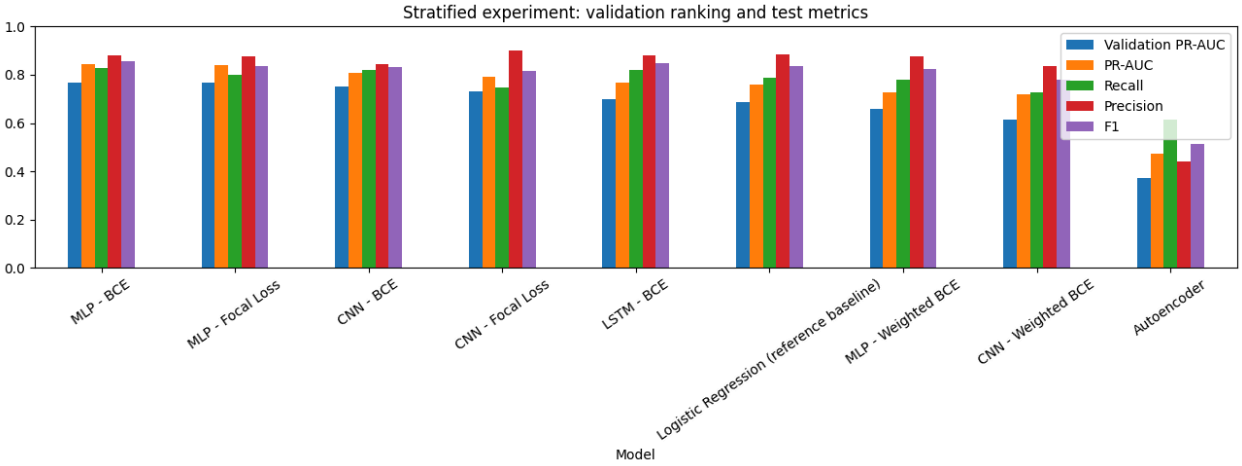
              precision    recall  f1-score   support

           0     0.9997    0.9998    0.9998     56863
           1     0.8817    0.8283    0.8542        99

    accuracy                         0.9995     56962
   macro avg     0.9407    0.9140    0.9270     56962
weighted avg     0.9995    0.9995    0.9995     56962

Confusion matrix:
 [[56852    11]
 [   17    82]]
```





11. SHAP explanation: all test frauds plus random genuine transactions

```
import shap

Xtr,Xv,Xt,ytr,yv,yt,feature_names = original_run["arrays"]
best_mlp_variants = {
    "MLP - BCE": original_run["models"]["MLP - BCE"],
    "MLP - Weighted BCE": original_run["models"]["MLP - Weighted BCE"],
    "MLP - Focal Loss": original_run["models"]["MLP - Focal Loss"]
}
# Select the MLP variant by validation PR-AUC only.
best_mlp_name = max(best_mlp_variants,
    key=lambda n: average_precision_score(yv,
        best_mlp_variants[n].predict(Xv,batch_size=BATCH_SIZE,verbose=0).ravel()))
best_mlp = best_mlp_variants[best_mlp_name]
fraud_idx = np.where(yt==1)[0]
genuine_idx = np.where(yt==0)[0]
rng = np.random.default_rng(SEED)
genuine_sample = rng.choice(genuine_idx, size=min(50,len(genuine_idx)), replace=False)
explain_idx = np.concatenate([fraud_idx,genuine_sample])
explain_data = Xt[explain_idx]
background_idx = rng.choice(len(Xtr), size=min(40,len(Xtr)), replace=False)
background = Xtr[background_idx]
```